

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed Features

Notable Concerns

The Case for User Controlled Inlining

Why not go for broke and jump straight into granular AST node manipulation?

Invocable and the Dangers of using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

Doc. No.:	P1221R0
Date:	2018-10-03
Reply To:	Jason Rice ricejasonf@gmail.com
Title:	Parametric Expressions
Audience:	Evolution Working Group

Parametric Expressions

Abstract

The purpose of this paper is to propose a function-like construct that creates an expression inline that can be used in idiomatic code with faster compile times.

Consider the following declaration syntax:

```
using add(auto a, auto b) {  
    return a + b;  
}
```

Here is a function-like declaration that has the same behaviour as a function without creating a function type, still has its own scope.

```
int main() {  
    int i = 40;  
    int x = add(i++, 2);  
}  
  
// the same as  
  
int main() {  
    int i = 40;  
    int x = ({  
        auto&& a = i++;  
        auto&& b = 2;  
        a + b; // result of expression  
    });  
}
```

Note that each input expression is bound to a variable resulting in evaluation that is consistent with normal call expressions. User controlled evaluation is addressed in *Using Parameters*.

Function templates as we have them now are loaded with features such as overloading, constraints, type deduction, etc.. When called not only do we get an expensive template instantiation but also potentially large amounts of extraneous code generation all for functions we wouldn't call otherwise unless we expected the optimizer to inline them anyways.

With this tool we want to give programmers the ability to allow this inlining at an earlier stage, but since the inlining is not guaranteed and in the context of where it is called we can do things not currently possible with normal functions by creating a special type for each invocation.

This ability to transform expressions is similar to the way we use type aliases, but here we can work with constructs that have run-time effects as seen in Fusion/Hana style metaprogramming. Additionally, since this is a language feature, it could potentially see compile-time performance that is comparable to pure-type template metaprogramming.

Constexpr Parameters

Since the invocation is inlined we easily get constexpr parameters without generating a complicated symbol for each prototype.

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed Features

Notable Concerns

The Case for User Controlled Inlining

Why not go for broke and jump straight into granular AST node manipulation?

Invocable and the Dangers of using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

```
using to_int_c(constexpr auto x) {
    return std::integral_constant<int, x>{};
}

int main() {
    constexpr int x = 42;
    std::integral_constant<int, 42> foo = to_int_c(42);
}
```

Using Parameters

With `using` as a specifier, we allow the user to take control of evaluation with macro-like semantics.

```
using if_(using auto cond, using auto x, using auto y) {
    if (cond)
        return x;
    else
        return y;
}

int main() {
    if_(
        true,
        print("Hello, world!"),
        print("not evaluated so nothing gets printed")
    );
}
```

This has some compelling use cases for creating idiomatic interfaces:

```
// customized conjunction and disjunction with short-circuit evaluation
```

```
Foo x = x || new Foo();
```

```
auto y = do_something() && maybe_do_something_else();
```

```
// constexpr ternary expression
```

```
auto x = constexpr_if(true, int{42}, std::make_unique<int>(42));
```

```
// debug logging that eliminates evaluation in production (like macros)
```

```
namespace log {
    struct null_stream {
        using operator<<(using auto) {
            return null_stream{};
        }
    };

    using warn() {
        if constexpr (log_level == Warning)
            return std::wcerr;
        else
            return null_stream{};
    }
}
```

```
foo > 42 && log::warn << "Foo is too large: " << foo << '\n';
// ^ implicitly invokes nullary call
```

Concise Forwarding

With this feature we can implement a forwarding operator that is more concise than the existing `std::forward`

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed Features

Notable Concerns

The Case for User Controlled Inlining

Why not go for broke and jump straight into granular AST node manipulation?

Invocable and the Dangers of using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

```
using fwd(using auto x) {  
    return static_cast<decltype(x)&&>(x);  
}  
  
auto old_f = [](auto&& x) { return std::forward<decltype(x)>(x); };  
auto new_f = [](auto&& x) { return fwd(x); };
```

Operations on Parameter Packs

We can allow unexpanded parameter packs as inputs and avoid creating tuples in many cases.

```
template <typename ...Xs>  
auto foo(Xs... xs) {  
    auto fifth_element = pack::get<5>(xs);  
  
    auto bar = pack::apply(f, pack::drop_front(xs));  
}
```

Composing Operations

Function calls add overhead, and while they are usually inlined by the optimizer, it is still generally avoided as significant compile-time overhead.

Consider the pattern of having functions return the result of other functions:

```
template <typename T>  
auto calc_a(T input) {  
    int on_the_stack = input + 42;  
    return calc_b(on_the_stack);  
}  
  
template <typename T>  
auto calc_b(T& on_the_stack) {  
    return calc_c(input + 1);  
}
```

We can decouple these functions by using a generic continuation interface.

```
do_  
    calc_a,  
    calc_b,  
    calc_c  
);
```

This is currently possible with normal functions, but it adds a bunch of noise to the call stack which makes debugging difficult, and the additional function calls make more work for the compiler which adds up.

As a Member of a Class

Parametric expressions can be used as a member in conjunction with operator overloading to create an invocable

```
struct id_fn {  
    using operator()(auto x) {  
        return x;  
    }  
};  
  
id_fn{}(42);  
  
static_assert(std::is_invocable<id_fn, int>::value);
```

RAII Scope Elision

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed Features

Notable Concerns

The Case for User Controlled Inlining

Why not go for broke and jump straight into granular AST node manipulation?

Invocable and the Dangers of using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

Specific conditions allow us to generate an expression that does not require RAI and therefore allows the co substitute the call expression directly which can yield a prvalue and can also be used in SFINAE and other cc variable declarations and statements are not welcome.

```
using funcify(using auto f) {
    return [](auto&& ...x)
        noexcept(noexcept(decltype(f(std::forward<decltype(x)>(x)>(x))))))
        -> decltype(f(std::forward<decltype(x)>(x)>(x)...))
        { return f(std::forward<decltype(x)>(x)>(x)...); };
}

struct make_array_impl {
    using operator()(using auto ...x) {
        return std::array<std::common_type<decltype(x)>..., sizeof...(x)>{x...};
    }
};

constexpr auto make_array = funcify(make_array_impl{});
```

Comparison with Other Proposed Features

One of the primary benefits of this proposed feature is the compile-time performance and the fact that it has z the type system. That with operations on packs, there are no other proposals addressing this. However this p feature does have overlap with other proposed features concerning constexpr parameters and conditional ev

The paper *constexpr Function Parameters* (P1045R1) by David Stone proposes the `constexpr` specifier t normal function parameter declarations effectively making the parameter `constexpr`. The advantage that it l Parametric Expressions proposal is that it allows function overloading. While this is still highly desirable, it co costs which will affect compiler throughput. First, it requires overload resolution to check if an argument expre used in a constant expression. Second, each constexpr parameter would affect the type creating a new funct symbols that would have to be similar to anonymous classes for each permutation of input arguments. If this feasible, there is no conflict and the `constexpr` parameters proposed by this paper are consistent with Davi proposal and the feature proposed in this paper would be a useful augment for when overloading is not need

The paper *Towards A (Lazy) Forwarding Mechanism for C++* (P0927R0) by James Dennet and Geoff Romer sets out to add a feature to enable conditional evaluation of function parameters. The paper suggests the use similar to a lambda expression for the parameter declaration to specify laziness of evaluation which is a majo convention as far as variable declarations are concerned. This paper is in agreement with the motivations str solutions are in contrast. Having the same issues with creating a function prototype, the paper addresses the on the type system suggesting that lazy parameters be added to the type system which is high impact, comp opinion of this paper, unnecessary. This paper, however, is proposing the use of an existing keyword in the d specifier which is more in keeping with existing conventions with parameter declarations, and there is no imp system which makes it a better solution.

Notable Concerns

The Case for User Controlled Inlining

As far as function call inlining is concerned, optimizers bear a monopoly on this ability as far as what can be c standard C++. Abuse of this proposed feature could lead to creating code that might be suboptimal versus ha where the optimizer deduces that inlining should not occur.

However, there are cases where it would benefit the user to be able to prevent symbol bloat in the ABI, and v probably a compiler QOI issue, optimizing generic or template heavy code with `-Os` usually results in very la because the optimizer doesn't inline as aggressively. Allowing the user to surgically inline functions where it m can mitigate this.

Additionally inlining code on the compiler front end can bypass massive amounts of symbol generation and v optimizer for simple cases such as accessing the members of a large tuple. Compiler performance is no sma many go to great lengths to avoid libraries that, while providing interfaces that are more idiomatic and make c more maintainable, they add significant overhead to build times effectively killing productivity. Adding parame expressions would solve this.

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed Features

Notable Concerns

The Case for User Controlled Inlining

Why not go for broke and jump straight into granular AST node manipulation?

Invocable and the Dangers of using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

Why not go for broke and jump straight into granular AST node manipulation?

Some have suggested that we add something more akin to raw AST generation as is found in Rust macros. While it would be a powerful superset of the feature this paper is proposing, it comes with some issues. One, this would interface **significantly more complicated** and inaccessible to typical users. It also comes with the same problem of evaluation that preprocessor macros have in that it easily allows mistakes that result in duplicate evaluation of operations.

The interface proposed with *Parametric Expressions* is such that it defaults to consistent evaluation of input expressions by binding them to an actual variable declaration in the parameter list. If the author of a function wants to take control of evaluation they can do so via the `using` parameter declaration specifier and thus opt in explicitly and without changing the AST syntax.

That said, the advent of Reflection and Metaclasses may open the door to allowing more refined control over this paper's stance is that this feature is **forwards compatible** with the possibility of adding a `reflexpr` parameter declaration specifier to enable this superset.

Invocable and the Dangers of using Parameters

As previously stated, `using` parameters add macro-like semantics. While the pitfalls of macros are well understood, there is a particularly onerous edge case that is worth mentioning:

```
struct eval_twice_fn {
    using operator()(using auto x) {
        return std::pair(x, x);
    }
} inline constexpr eval_twice{};

auto result1 = eval_twice(my::string("foo"));
auto result2 = std::invoke(eval_twice, my::string("foo"));
```

Here the semantics are changed when used with `std::invoke`. Since `std::invoke` is a normal function, it takes a function object and binds it to a parameter that is perfectly forwarded. The forwarding expression would then be passed to the function and the result of the input would get moved twice. With the call expression the result is a pair of two distinct strings, each containing a pointer to their own "foo".

`std::Invokable` does not require that the call to `std::invoke` be equality preserving on its arguments so this is perfectly permissible. Regardless, this paper's stance is to allow these types of edge cases, and leave it to the discretion of the user as they are opting in explicitly via the `using` specifier. Attempts to prevent these would involve complex rules that would inadvertently rule out desirable use cases.

Impact on the Existing Standard

While there is zero impact on existing features in C++, the following additions are required:

1. Parsing the `using` keyword followed by an identifier or operator name followed by a left parentheses would be used to denote the declaration of a parametric expression disambiguating from other valid uses of the `using` keyword for declarations.
2. Allow the `constexpr` specifier in parameter declarations in the context of a parametric expression parameter declaration.
3. The `using` keyword is used as a declaration specifier for parameter declarations in the context of a parametric expression parameter list declaration.
4. The `auto` keyword is used as a placeholder for a constraint for parametric expression parameters and it is used to promote parameters as `auto&&`. This is more concise but a slight deviation from what C++ programmers expect.

The Rules

The following rules have been refined based on experience from the [reference implementation](#):

1. Declaration or invocation does not create a type. Overloading or ADL is not supported.

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed Features

Notable Concerns

The Case for User Controlled Inlining

Why not go for broke and jump straight into granular AST node manipulation?

Invocable and the Dangers of using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

2. The input parameter list may contain only one parameter pack in any position.
3. Input parameter type specifiers must be completely unconstrained and not specify a type.
 - Only `auto` is allowed as the type specifier. The idea here is to be forwards compatible with a concise specifying a constraint via the concepts language feature.
 - The other option is to allow specifying a type, but this is ugly and not useful as overloading is not supported.
4. Parameter declarations can have `constexpr` or `using` specifiers which change semantics:
 - With none of these specifiers the parameter is an rvalue reference initialized with the input expression is treated as an lvalue. (just like `auto&&`)
 - The `constexpr` specifier is a value initialized with an expression that must be a constant-expression.
 - The `using` specifier does not create a variable and the input expression is substituted anywhere the variable is used. (like a macro)
5. The input parameters may be unexpanded parameter packs.
 - This adds a bit more power for working with lists.
6. The definition is a compound statement that may have RAII scope when invoked.
 - A RAII scope is not created if there is only one statement that is a return statement, and all parameters are declared with the `using` specifier.
 - A RAII scope is created if there are any statements other than a single return statement or if there are parameter declarations without the `using` specifier.
 - Implicit `this` does not count as a variable declaration for the purposes of detecting if there should be a RAII scope.
7. If a RAII scope is created then invocation executes the compound statement as an expression with the return type determined by its return statements with the same RVO/NRVO guaranteed copy elision rules that apply to member functions.
8. If no RAII scope is created then invocation results in substituting the expression from the return statement.
9. As with deduced return types in functions, the return statements must all result in the same type not constant expressions omitted after static selection.
10. Recursion is not allowed
 - Recursion is not feasible, but we could at least use Odin Holmes' recursive alias pattern
11. Operator overloading is supported. (only as a member since ADL is not supported)
12. When defined as a member of a class, invocations have implicit `this`. An optional `const` qualifier after the parameter list can be used to restrict `this` to `const`.
 - This is consistent with the way member functions behave.
 - An object with a defined `using operator()` should be invocable.
13. As a member, the `static` keyword can be used to simply make it a free entity that is local to the owning class.
 - This is consistent with the way member functions behave.
14. The use of an identifier that refers to the name of a parametric expression, when used in an expression without parentheses (ie no parens), is implicitly promoted to a nullary call.
 - This is a nice-to-have as it can reduce syntactic noise and make nullary parametric expressions appear more like a variable.

Summary

With this we have a facility for writing more idiomatic code and with a significant improvement in compile-time performance having an impact that would rival what was seen with the advent of variadic templates.

This feature has the potential to have a huge impact on the ability to implement ranges, expression templates, metaprogramming libraries, and functional and generic code as well as replace the use of macros in a lot of code.

Parametric Expressions

Abstract

Constexpr Parameters

Using Parameters

Concise Forwarding

Operations on Parameter Packs

Composing Operations

As a Member of a Class

RAII Scope Elision

Comparison with Other Proposed
Features

Notable Concerns

The Case for User Controlled
Inlining

Why not go for broke and jump
straight into granular AST node
manipulation?

Invocable and the Dangers of
using Parameters

Impact on the Existing Standard

The Rules

Summary

Acknowledgements

would be really cool to get this in C++20 if it is not too late.

Thanks for looking at this!

Jason Rice

Acknowledgements

Special thanks to Louis Dionne and Miguel Ojeda who provided feedback and suggestions as well as others feedback on the mailing list.